

Customer Churn Risk Analysis & Segmentation — Project Report

1. Project Overview

Objective: Analyze subscription-based customer data to understand *why customers churn*, *how much revenue is at risk*, and *which customer segments are riskiest* — so the business can prioritize retention efforts.

Tools used: Python (pandas, numpy), SQLite (via `sqlite3`), Matplotlib, Seaborn, Jupyter Notebook.

Data source: An SQLite database (`customer_churn.db`) containing three tables:

Table	What it holds
<code>db_customer</code>	Demographics — customer ID, name, country, state, gender, date of birth, interests, pincode
<code>db_subscription</code>	Subscription/billing facts — plan type, contract type, monthly charges, CLTV, churn score, cancellation date/reason
<code>db_support</code>	Customer support interactions — complaint date, escalation flag, CSAT score, comments

The project follows the standard analytics pipeline: **Extract** → **Clean** → **Engineer Features** → **Analyze** → **Visualize** → **Segment** → **Report**. Below is a step-by-step walkthrough of every stage, what it does, why that method was chosen, and why the alternatives weren't.

2. Data Extraction (Loading from SQLite)

What it does: Connects to the SQLite `.db` file, queries `sqlite_master` to list all tables, then loops through each table name and loads it into its own pandas DataFrame (`df_db_customer` , `df_db_subscription` , `df_db_support`) using `globals()[f"df_{table_name}"] = df`.

Why this method: - Querying `sqlite_master` first (instead of hardcoding table names) makes the loader generic — it adapts automatically if tables are added or renamed. - Using `pd.read_sql()` pulls data directly into pandas, which is far more convenient for cleaning/analysis than working with raw `cursor.fetchall()` tuples. - `globals()` dynamically creates one DataFrame per table without repetitive manual assignment — useful for a small, known number of tables.

Why not other approaches: - A single `SELECT *` join at the database level was avoided at this stage because the tables have different granularities (customer = 1 row/customer, support = multiple rows/customer for repeat complaints). Joining too early — before deduplication — would have silently inflated row counts (this is exactly the mistake caught and fixed in Section 4). - `globals()` is a shortcut that works for a handful of tables; in a production pipeline, a dictionary of DataFrames (`tables = {name: df}`) would be safer since `globals()` pollutes the namespace and doesn't scale. - The connection is explicitly closed with `conn.close()` after loading — necessary because SQLite locks the file while a connection is open; leaving it open would block later write access to the same file.

3. Data Cleaning

Each cleaning action targets a specific, identified data-quality problem — this is more defensible than blanket cleaning, because every change is traceable to a reason.

3.1 Rename column (`name` → `customer_name`)

What: `df_db_customer.rename(columns={'name': 'customer_name'})` **Why:** `name` is a vague, easily-confused column label (and a near-reserved word in SQL/pandas contexts). Renaming to `customer_name` avoids ambiguity once this table is merged with others later. **Why not leave it:** After merging three tables, an unqualified `name` column becomes unreadable in a wide DataFrame — better to disambiguate early than to rename post-merge.

3.2 Drop irrelevant columns (`interests` , `pincode`)

What: Drops both columns from `df_db_customer`. **Why:** `.info()` showed `interests` had only 4/21 non-null values and `pincode` had **0/21** non-null values — i.e., pincode carried zero information. Columns that are almost entirely missing add noise without adding signal, and they aren't relevant to churn risk. **Why not impute instead:** Imputing a column that is 100% missing (pincode) is meaningless — there's no data to learn a pattern from. For `interests` (81% missing), imputing would fabricate values for a variable not hypothesized to drive churn, so dropping was more honest than guessing.

3.3 Fix data type: `dob` → `datetime`

What: `pd.to_datetime(df_db_customer['dob'])` **Why:** Dates loaded from SQLite/CSV often come in as strings (`object` / `str` dtype). Converting to `datetime64` is required to do date arithmetic later (e.g., computing customer age). **Why not keep as string:** String dates can't be subtracted, compared, or have `.dt` accessor methods (year, days, etc.) applied — the age and tenure calculations in Section 5 would be impossible without this conversion.

3.4 Standardize categorical values (gender)

What: `.replace({'Men': 'Male', 'Women': 'Female'})` **Why:** `.unique()` on the raw column would have shown four labels (Male , Female , Men , Women) representing only two actual categories — a classic **data inconsistency** issue, likely from different data-entry sources. Standardizing prevents the same real-world group from being split across two rows in any groupby/aggregation. **Why not leave as-is or use fuzzy matching:** With only two clearly-mapped synonym pairs, a simple `.replace()` dictionary is the most transparent, deterministic fix. Fuzzy string matching (e.g., `fuzzywuzzy`) would be overkill and riskier (it can silently mis-map values) for a case this small and unambiguous.

3.5 Fix missing values (country)

What: Builds a `state → country` lookup dictionary from rows that *do* have a country, then uses `.fillna()` with `.map()` to fill the missing `country` values based on each row's `state`. **Why:** Country is missing for only 3 rows, but `state` is fully populated and each state uniquely maps to one country (e.g., "Kathmandu" → Nepal, "Maharashtra" → India) — a **logical, rule-based imputation** rather than a statistical guess. **Why not drop the rows or fill with mode/"Unknown":** - Dropping would lose 3 of 21 customers (~14% of the dataset) — too costly on an already small dataset. - Filling with the overall mode (e.g., "India") would be wrong for the row where `state = Kathmandu` , since that's clearly Nepal — mode-imputation ignores information the dataset already contains. - Because state→country is a deterministic real-world relationship (not a statistical correlation), mapping is strictly more accurate than any averaging technique.

3.6 Deduplicate the support table before merging

What: Sorts `db_support` by `complaint_date` , then `drop_duplicates('customerid', keep='last')` — keeping only each customer's most recent complaint — after first computing `complaint_count` via `groupby('customerid').transform('count')`. **Why:** The support table has multiple rows per customer (9 rows, only 7 unique customers) because customers can log more than one complaint. If left as-is, a `merge()` on `customerid` would create a **fan-out join** — duplicating subscription/customer rows for every extra complaint a customer has (this exact bug was caught: `df.shape` was (23, 20) before the fix vs. the correct (21, 21) after). **Why not keep all complaint rows / use first instead of last :** - Keeping all rows breaks the "one row = one customer" grain the rest of the analysis depends on (churn rate, ARPU, etc. would be double-counted for repeat complainers). - `complaint_count` is calculated *before* deduplication specifically to preserve the "how many times did they complain" signal — dropping duplicates would otherwise destroy that information. - `keep='last'` (most recent complaint) was chosen over `keep='first'` because the most recent interaction is more relevant to current churn risk than the oldest one.

4. Merging the Three Tables

What it does: Left-joins `db_subscription` (the base, since every customer has exactly one subscription record) with `db_customer` and the de-duplicated `db_support` , both `on='customerid'` .

Why subscription as the base table with how='left' : - Every customer has a subscription, but not every customer has filed a support ticket. A **left join** from subscription preserves all 21 customers regardless of whether they ever contacted support — an inner join would silently drop customers with no support history, which would bias churn analysis toward only "vocal" customers. - `df.shape` was checked immediately after the merge ((21, 21)) to confirm the row count matched the expected number of unique customers — a quick sanity check that catches join bugs (this is exactly how the earlier duplication bug in Section 3.6 was discovered).

Why not merge all three tables in one earlier step: As noted above, merging before deduplicating the support table would have caused row duplication. Cleaning → then merging is the correct sequence; merging → then cleaning risks corrupting the very rows you're trying to fix.

Output: The merged table is exported once via `df.to_csv(...)` — creating a clean, analysis-ready snapshot so downstream steps don't need to re-run the extraction/merge every time.

5. Feature Engineering

New columns are engineered specifically to answer churn-related business questions that don't exist as raw fields.

New feature	How it's built	Why it's needed
churn_flag	1 if <code>cancellation_date</code> is not null, else 0 (via <code>np.where</code>)	Converts a date field into a binary label — the target variable for all churn-rate calculations. Churn isn't stored explicitly in the source data; it has to be <i>derived</i> from "did this customer cancel."
customer_age	Current year – birth year (from <code>dob</code>)	Age is a common churn-risk covariate (younger vs. older customers often churn differently); not present in raw data.
tenure_days	If churned: <code>cancellation_date - subscription_start_date</code> ; else: <code>today - subscription_start_date</code> (via <code>np.where</code>)	Measures how long a customer has stayed. Using <code>np.where</code> (rather than one formula) is necessary because active customers have no cancellation date — "tenure so far" and "tenure until churn" need different formulas.
churn_risk (Low/Med/High)	Bucketed from <code>churn_score</code> using <code>np.select</code> with cutoffs <code><50</code> , <code>50–69</code> , <code>≥70</code>	Turns a continuous score into a business-friendly categorical label — easier for stakeholders to act on ("target all High-risk customers") than raw numeric scores.
complaint_count	Count of complaints per customer (computed	Captures complaint <i>frequency</i> , a signal otherwise lost once duplicate support rows are

Why `np.where` / `np.select` instead of `.apply()` with a Python function: Both are **vectorized** — they operate on the whole column at once using compiled NumPy code, which is significantly faster than row-by-row `.apply()`, and they scale to larger datasets without a performance cliff. For simple threshold/conditional logic like this, vectorized functions are the standard, idiomatic pandas approach.

6. KPI / Business-Metric Analysis

Each of the following one-line metrics answers a specific business question. This is the analytical core of the report — the numbers stakeholders would actually see in a dashboard.

Metric	Formula	Result	What it tells the business
Churn Rate	<code>mean(churn_flag) × 100</code>	28.57%	Nearly 3 in 10 customers have cancelled — a high churn rate that justifies deeper investigation.
Retention Rate	<code>100 – churn rate</code>	71.43%	The flip side of churn; useful for framing results positively to stakeholders.
Churn by Plan Type	<code>groupby(plan_type) [churn_flag].mean()</code>	Basic 60% / Standard 22.2% / Premium 14.3%	Basic-plan customers churn 4× more than Premium — plan tier is a strong churn driver, likely tied to price sensitivity or lower perceived value.
Churn by State/Subscription Source	<code>groupby + .agg()</code> with named aggregations	Varies by state; Referral -sourced customers churn far more (5/6 churned) than Organic (0/9 churned)	Acquisition channel matters — referral customers may have weaker product-market fit or lower initial intent than organically-acquired ones.
ARPU (Avg Revenue Per User)	<code>mean(monthly_charges)</code>	\$18.85	Baseline revenue benchmark; needed to compute revenue impact of churn.
Avg Tenure	<code>mean(tenure_days)</code>	~1,530 days	Shows typical customer lifespan; useful for CLTV modeling.
Revenue at Risk	<code>sum(monthly_charges) where churn_flag ==1</code>	\$73.94/month	Direct dollar cost of churn — the number that turns an analytics finding into a business case.
Escalation Rate	% of support tickets marked Y for escalation	19.05%	Flags how often support issues become serious enough to escalate — a leading indicator of dissatisfaction.
Avg Complaints per User	total complaints / unique customers	0.43	Overall complaint density across the base.
Escalation-Churn Correlation	<code>.corr()</code> between <code>escalations</code> and <code>churn_flag</code>	NaN (see note below)	Intended to test whether escalated complaints predict churn.

Why named aggregation (`.agg(total_revenue=('monthly_charges','sum'), ...)`) instead of multiple separate `.groupby()` calls: It computes several different statistics (sum, count, mean) across different columns in a single pass, returning one clean, readable table — far more efficient and readable than running `.groupby().sum()`, `.groupby().count()`, `.groupby().mean()` separately and merging the results.

On the NaN correlation result — what happened and why: At the point this was calculated, `df['escalations']` still contained `'Y' / NaN` string values, not numeric 0/1 — `.corr()` can't compute a Pearson correlation on non-numeric/mostly-missing data, so it returned `NaN`. This is flagged in-code (`# df['escalations'] = np.where(...)` is commented out) as a step that was planned but not completed before the correlation was run. **This is a genuine gap in the project** — see Section 9 (Limitations) for the fix.

7. Data Visualization

A `df_visual` copy is made specifically for plotting, so type conversions and encodings needed only for charts don't contaminate the analysis-ready `df`.

Why copy the DataFrame first: Visualization often requires converting dtypes (e.g., turning `churn_flag` into a clean 0/1 int, converting categories to numeric codes for a heatmap) that aren't appropriate for the main analytical DataFrame. Working on a copy isolates these transformations and avoids accidentally overwriting analysis results — this is also *why* the `churn_flag` column had to be recomputed in `df_visual` at Cell 64 (its dtype had drifted to all-`NaN / float64` after the earlier merge, a bug that was caught via `.value_counts()` and fixed with `.notna().astype(int)`).

Chart	Library	What it shows	Why this chart type
Monthly Churn Trend (line chart)	Matplotlib	Cancellations grouped by month	A line chart is the standard choice for a time-series trend — it makes upward/downward momentum in churn immediately visible, which a bar chart or table would obscure.

Churn Rate by Plan Type (bar chart)	Matplotlib	Mean churn flag per plan, colored per category	Bar charts are the right tool for comparing a metric across a small number of discrete categories (3 plan types) — length is easy to compare visually.
Churn Rate by State (bar chart)	Matplotlib, <code>plt.cm.Set2</code> colormap	Mean churn per state	Same reasoning as above, extended to more categories; a colormap (rather than manually listing colors) scales automatically to however many states are in the data.
Correlation Heatmap	Seaborn	Pairwise correlation between numeric features	Heatmaps let you scan many pairwise relationships (e.g., churn_score vs. tenure, escalations vs. churn) at once — far faster than checking <code>.corr()</code> values for every pair individually. Categorical columns were label-encoded first (<code>.astype('category').cat.codes</code>) because <code>.corr()</code> only works on numeric data.
Pairplot	Seaborn	Scatterplots + distributions for every numeric feature pair	Useful as an exploratory "first look" — surfaces obvious clusters, outliers, or separations between churned/non-churned customers across many variable combinations at once, without writing one scatterplot per pair manually.
Catplot / FacetGrid (<code>plan_type</code> vs <code>monthly_charges</code> , split by <code>gender</code> and <code>churn_risk</code>)	Seaborn	Multi-dimensional comparison in one figure	A facet grid lets you compare 4 variables at once (x, y, hue, and column-facet) without needing 2D or 3D charts that would be hard to read — each risk tier gets its own panel, each gender its own color.

Why label-encode by simple priority order (`Basic < Standard < Premium` , `Monthly < Annual` , `Low < Med < High`) rather than one-hot encoding: These categories have a natural **ordinal** ranking (there's a real "low-to-high" order), so preserving that order in a single numeric column keeps correlation and ranking analysis meaningful. One-hot encoding would create separate binary columns per category, which is appropriate for *unordered* (nominal) categories in machine-learning models, but would break the ability to read "higher number = more premium/more risk" directly off a correlation heatmap — the wrong tool for this exploratory, ordinal use case.

8. Pivot Tables

What it does: Uses `pd.pivot_table()` to summarize churn rate, customer count, and revenue by `plan_type` in a single compact table (e.g., Basic: 60% churn / 5 customers / \$52.95 revenue).

Why pivot tables here instead of repeating `groupby()` : A pivot table combines multiple aggregations (`mean` , `sum` , `nunique`) across multiple metrics into one summary view in a single call — this is exactly the kind of "cross-tab summary" pivot tables are designed for, and it mirrors the kind of table a business stakeholder would expect to see in Excel or a BI tool.

9. A Separate SQL-in-Python Demo Section

The final part of the notebook (creating a `users` table, inserting rows, running a `GROUP BY` aggregation query) is a **standalone practice exercise** in using `sqlite3` + `pandas.read_sql()` together — it uses a different, unrelated toy dataset (`Madhav` , `Rishabh` , `Vishakha`) and isn't connected to the churn analysis. It demonstrates the same read/write/query pattern used to load the original churn tables in Section 2, but isn't part of the churn findings. **Recommendation:** keep this in a separate notebook/section, since mixing it into the churn report can confuse a reader into thinking it's part of the business analysis.

10. Key Findings Summary

- Churn rate is high (28.6%)**, with **\$73.94/month in revenue at risk** from already-churned customers.
- Plan tier is the strongest churn driver found:** Basic-plan churn (60%) is over 4× Premium-plan churn (14.3%).
- Acquisition channel matters:** Referral-sourced customers churn far more than Organic ones — worth investigating referral program quality/targeting.
- Escalations and complaints are relatively rare** (19% escalation rate, 0.43 complaints/user) but the intended correlation test between escalations and churn wasn't completed correctly (see Limitations).
- Customers were successfully segmented into **Low / Medium / High churn-risk tiers**, giving the business a ready-made target list for retention campaigns (start with the High tier).

11. Limitations & Recommended Fixes

- Very small sample size (21 customers).** Every percentage above (churn rate, churn-by-plan, etc.) is based on single-digit counts per group — e.g., "Basic churn = 60%" is 3 out of 5 customers. These figures should be treated as directional, not statistically robust, until validated on a larger dataset.

- **Escalation-churn correlation returned NaN**. The `escalations` column needed to be converted from `'Y' / NaN` strings to numeric (0/1) *before* calling `.corr()` — the code to do this exists but is commented out. Fixing this (`df['escalations'] = np.where(df['escalations']=='Y', 1, 0)` before the `.corr()` call) would let this analysis actually run.
- **No formal segmentation model**. The current "Low/Med/High" risk buckets come from fixed cutoffs on `churn_score` , not from a clustering or predictive model (e.g., K-Means, logistic regression). This is a reasonable, transparent starting point, but a next iteration could validate/replace it with a model trained on the engineered features (tenure, complaints, plan type, escalations).
- **churn_score 's own origin is unclear** — it's used as the basis for risk segmentation, but the notebook doesn't show how it was originally calculated (it comes pre-populated in `db_subscription`). Understanding that formula would clarify what's actually driving the risk tiers.